

Dynamic Programming – Basic Problems

Dice Throw Problem | Binomial Coefficient | Assembly Line Scheduling

1. Dice Throw Problem

Question 1: Online Game Reward Unlock

Aim:

To determine the number of ways a player can unlock a reward in an online game by rolling 3 six-faced dice and obtaining a total score of exactly 8, using Dynamic Programming.

Algorithm:

1. Read the number of dice N , the number of faces per die M , and the target sum S .
2. Create a 2-D table dp of size $(N+1) \times (S+1)$, initialised to 0, where $dp[d][s]$ is the number of ways to obtain sum s using d dice.
3. Set the base case $dp[0][0] = 1$ (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S , sum $dp[d-1][s-f]$ for every face value f from 1 to M where $s-f$ is non-negative.
5. The value $dp[N][S]$ gives the total number of ways to obtain the target sum with N dice.
6. Print $dp[N][S]$ as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(3, 6, 8))
```

Input:

```
Number of Dice    = 3
Faces per Dice    = 6
Target Score/Sum  = 8
```

Output:

```
Number of ways = 21
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 2: Board Game Tournament

Aim:

To calculate the number of possible outcomes in a board game tournament where a player rolls 2 six-faced dice and must obtain a total score of 7 to qualify for the next round.

Algorithm:

1. Read the number of dice N , the number of faces per die M , and the target sum S .

2. Create a 2-D table dp of size (N+1) x (S+1), initialised to 0, where dp[d][s] is the number of ways to obtain sum s using d dice.
3. Set the base case dp[0][0] = 1 (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S, sum dp[d-1][s-f] for every face value f from 1 to M where s-f is non-negative.
5. The value dp[N][S] gives the total number of ways to obtain the target sum with N dice.
6. Print dp[N][S] as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(2, 6, 7))
```

Input:

```
Number of Dice    = 2
Faces per Dice    = 6
Target Score/Sum  = 7
```

Output:

```
Number of ways = 6
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 3: Warehouse Robot Navigation

Aim:

To determine the number of possible movement sequences of a warehouse robot that reaches its destination when the sum of 4 throws of a 4-faced die equals 10.

Algorithm:

1. Read the number of dice N, the number of faces per die M, and the target sum S.
2. Create a 2-D table dp of size (N+1) x (S+1), initialised to 0, where dp[d][s] is the number of ways to obtain sum s using d dice.
3. Set the base case dp[0][0] = 1 (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S, sum dp[d-1][s-f] for every face value f from 1 to M where s-f is non-negative.
5. The value dp[N][S] gives the total number of ways to obtain the target sum with N dice.
6. Print dp[N][S] as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
```

```

        if t - f >= 0:
            total += dp[d - 1][t - f]
        dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(4, 4, 10))

```

Input:

```

Number of Dice    = 4
Faces per Dice    = 4
Target Score/Sum  = 10

```

Output:

```

Number of ways = 44

```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 4: Casino Reward Calculation

Aim:

To calculate the total number of winning combinations for a casino reward given to customers who obtain exactly 12 points by rolling 3 six-faced dice.

Algorithm:

1. Read the number of dice N, the number of faces per die M, and the target sum S.
2. Create a 2-D table dp of size (N+1) x (S+1), initialised to 0, where dp[d][s] is the number of ways to obtain sum s using d dice.
3. Set the base case dp[0][0] = 1 (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S, sum dp[d-1][s-f] for every face value f from 1 to M where s-f is non-negative.
5. The value dp[N][S] gives the total number of ways to obtain the target sum with N dice.
6. Print dp[N][S] as the final answer.

Program (Python Code):

```

def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(3, 6, 12))

```

Input:

```

Number of Dice    = 3
Faces per Dice    = 6
Target Score/Sum  = 12

```

Output:

```

Number of ways = 25

```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 5: Quiz Competition Scoring

Aim:

To find the number of valid scoring combinations in a quiz application where bonus points are awarded if the sum of 5 six-faced dice rolls equals 15.

Algorithm:

1. Read the number of dice N , the number of faces per die M , and the target sum S .
2. Create a 2-D table dp of size $(N+1) \times (S+1)$, initialised to 0, where $dp[d][s]$ is the number of ways to obtain sum s using d dice.
3. Set the base case $dp[0][0] = 1$ (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S , sum $dp[d-1][s-f]$ for every face value f from 1 to M where $s-f$ is non-negative.
5. The value $dp[N][S]$ gives the total number of ways to obtain the target sum with N dice.
6. Print $dp[N][S]$ as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(5, 6, 15))
```

Input:

```
Number of Dice    = 5
Faces per Dice    = 6
Target Score/Sum  = 15
```

Output:

```
Number of ways = 651
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 6: Sports Prediction System

Aim:

To determine the number of ways a predicted score of 9 can be obtained using 3 five-faced dice.

Algorithm:

1. Read the number of dice N , the number of faces per die M , and the target sum S .
2. Create a 2-D table dp of size $(N+1) \times (S+1)$, initialised to 0, where $dp[d][s]$ is the number of ways to obtain sum s using d dice.
3. Set the base case $dp[0][0] = 1$ (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S , sum $dp[d-1][s-f]$ for every face value f from 1 to M where $s-f$ is non-negative.

5. The value $dp[N][S]$ gives the total number of ways to obtain the target sum with N dice.
6. Print $dp[N][S]$ as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(3, 5, 9))
```

Input:

```
Number of Dice    = 3
Faces per Dice    = 5
Target Score/Sum  = 9
```

Output:

```
Number of ways = 19
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 7: Lottery Simulation System

Aim:

To find the number of possible combinations that result in a sum of 11 using a lottery simulation system with 2 ten-faced dice.

Algorithm:

1. Read the number of dice N, the number of faces per die M, and the target sum S.
2. Create a 2-D table dp of size $(N+1) \times (S+1)$, initialised to 0, where $dp[d][s]$ is the number of ways to obtain sum s using d dice.
3. Set the base case $dp[0][0] = 1$ (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S, sum $dp[d-1][s-f]$ for every face value f from 1 to M where s-f is non-negative.
5. The value $dp[N][S]$ gives the total number of ways to obtain the target sum with N dice.
6. Print $dp[N][S]$ as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
```

```
print("Number of ways =", dice_ways(2, 10, 11))
```

Input:

```
Number of Dice    = 2
Faces per Dice    = 10
Target Score/Sum  = 11
```

Output:

```
Number of ways = 10
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 8: Generalised N-Dice, M-Face Sum Counter

Aim:

To write a general program that finds the number of ways to obtain a given sum using N dice, each having M faces.

Algorithm:

1. Read the number of dice N, the number of faces per die M, and the target sum S.
2. Create a 2-D table dp of size (N+1) x (S+1), initialised to 0, where dp[d][s] is the number of ways to obtain sum s using d dice.
3. Set the base case dp[0][0] = 1 (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S, sum dp[d-1][s-f] for every face value f from 1 to M where s-f is non-negative.
5. The value dp[N][S] gives the total number of ways to obtain the target sum with N dice.
6. Print dp[N][S] as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(2, 6, 7))
```

Input:

```
Number of Dice    = 2
Faces per Dice    = 6
Target Score/Sum  = 7
```

Output:

```
Number of ways = 6
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 9: Generalised Target-Score Counter

Aim:

To determine the number of ways to obtain a target score using multiple dice, for arbitrary values of dice count, faces, and target.

Algorithm:

1. Read the number of dice N, the number of faces per die M, and the target sum S.
2. Create a 2-D table dp of size (N+1) x (S+1), initialised to 0, where dp[d][s] is the number of ways to obtain sum s using d dice.
3. Set the base case dp[0][0] = 1 (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S, sum dp[d-1][s-f] for every face value f from 1 to M where s-f is non-negative.
5. The value dp[N][S] gives the total number of ways to obtain the target sum with N dice.
6. Print dp[N][S] as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(3, 6, 8))
```

Input:

```
Number of Dice    = 3
Faces per Dice    = 6
Target Score/Sum  = 8
```

Output:

```
Number of ways = 21
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 10: Outcome Counter for a Specified Sum

Aim:

To count the possible outcomes that produce a specified sum when a given number of dice are rolled together.

Algorithm:

1. Read the number of dice N, the number of faces per die M, and the target sum S.
2. Create a 2-D table dp of size (N+1) x (S+1), initialised to 0, where dp[d][s] is the number of ways to obtain sum s using d dice.
3. Set the base case dp[0][0] = 1 (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S, sum dp[d-1][s-f] for every face value f from 1 to M where s-f is non-negative.
5. The value dp[N][S] gives the total number of ways to obtain the target sum with N dice.
6. Print dp[N][S] as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(4, 6, 10))
```

Input:

```
Number of Dice    = 4
Faces per Dice    = 6
Target Score/Sum  = 10
```

Output:

```
Number of ways = 80
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 11: Dice Combination Finder

Aim:

To find the number of possible dice combinations that yield a required score, using 2 dice with 4 faces each.

Algorithm:

1. Read the number of dice N, the number of faces per die M, and the target sum S.
2. Create a 2-D table dp of size (N+1) x (S+1), initialised to 0, where dp[d][s] is the number of ways to obtain sum s using d dice.
3. Set the base case dp[0][0] = 1 (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S, sum dp[d-1][s-f] for every face value f from 1 to M where s-f is non-negative.
5. The value dp[N][S] gives the total number of ways to obtain the target sum with N dice.
6. Print dp[N][S] as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]
    dp[0][0] = 1
    for d in range(1, num_dice + 1):
        for t in range(1, target + 1):
            total = 0
            for f in range(1, faces + 1):
                if t - f >= 0:
                    total += dp[d - 1][t - f]
            dp[d][t] = total
    return dp[num_dice][target]

if __name__ == "__main__":
    print("Number of ways =", dice_ways(2, 4, 5))
```

Input:

```
Number of Dice    = 2
```


Faces per Dice = 4
Target Score/Sum = 5

Output:

Number of ways = 4

Result:

The program was executed successfully and the output was verified to be correct for the given input.

2. Binomial Coefficient

Question 1: Dice-Throw Target-Sum Recap

Aim:

To determine the number of ways to obtain a target sum using dice throws, applying the same Dynamic Programming table-building technique used for the Binomial Coefficient problems that follow.

Algorithm:

1. Read the number of dice N, the number of faces per die M, and the target sum S.
2. Create a 2-D table dp of size (N+1) x (S+1), initialised to 0, where dp[d][s] is the number of ways to obtain sum s using d dice.
3. Set the base case dp[0][0] = 1 (zero dice give a sum of zero in exactly one way).
4. For each number of dice d from 1 to N and each target sum s from 1 to S, sum dp[d-1][s-f] for every face value f from 1 to M where s-f is non-negative.
5. The value dp[N][S] gives the total number of ways to obtain the target sum with N dice.
6. Print dp[N][S] as the final answer.

Program (Python Code):

```
def dice_ways(num_dice, faces, target):  
    dp = [[0] * (target + 1) for _ in range(num_dice + 1)]  
    dp[0][0] = 1  
    for d in range(1, num_dice + 1):  
        for t in range(1, target + 1):  
            total = 0  
            for f in range(1, faces + 1):  
                if t - f >= 0:  
                    total += dp[d - 1][t - f]  
            dp[d][t] = total  
    return dp[num_dice][target]  
  
if __name__ == "__main__":  
    print("Number of ways =", dice_ways(3, 4, 7))
```

Input:

Number of Dice = 3
Faces per Dice = 4
Target Score/Sum = 7

Output:

Number of ways = 12

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 1: Basic Binomial Coefficient Calculator

Aim:

To calculate the Binomial Coefficient $C(n, r)$ for given values of n and r using Dynamic Programming.

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size $(n+1) \times (r+1)$, initialised to 0.
3. For each row i from 0 to n and each column j from 0 to $\min(i, r)$: if j is 0 or j equals i , set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```
def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(5,2) =".format(n=5, r=2), binomial_coefficient(5, 2))
```

Input:

n (Total items) = 5
 r (Items required) = 2

Output:

$C(5, 2) = 10$

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 2: Item Selection Counter

Aim:

To find the number of ways to select r items from a group of n items using Dynamic Programming.

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size $(n+1) \times (r+1)$, initialised to 0.
3. For each row i from 0 to n and each column j from 0 to $\min(i, r)$: if j is 0 or j equals i , set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```
def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
```

```

        else:
            C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(8,3) =".format(n=8, r=3), binomial_coefficient(8, 3))

```

Input:

```

n (Total items) = 8
r (Items required) = 3

```

Output:

```

C(8,3) = 56

```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 3: DP-Based Binomial Coefficient Computation

Aim:

To compute the Binomial Coefficient using Dynamic Programming for larger values of n and r.

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size (n+1) x (r+1), initialised to 0.
3. For each row i from 0 to n and each column j from 0 to min(i, r): if j is 0 or j equals i, set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```

def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(10,4) =".format(n=10, r=4), binomial_coefficient(10, 4))

```

Input:

```

n (Total items) = 10
r (Items required) = 4

```

Output:

```

C(10,4) = 210

```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 4: Committee Formation Counter

Aim:

To determine the number of possible committees of a given size that can be formed from a group of members.

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size $(n+1) \times (r+1)$, initialised to 0.
3. For each row i from 0 to n and each column j from 0 to $\min(i, r)$: if j is 0 or j equals i , set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```
def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(12,5) = ".format(n=12, r=5), binomial_coefficient(12, 5))
```

Input:

n (Total items) = 12
 r (Items required) = 5

Output:

$C(12, 5) = 792$

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 5: General Combination Value Calculator

Aim:

To calculate the combination value $C(n, r)$ for given n and r .

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size $(n+1) \times (r+1)$, initialised to 0.
3. For each row i from 0 to n and each column j from 0 to $\min(i, r)$: if j is 0 or j equals i , set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```
def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
```

```

        C[i][j] = 1
    else:
        C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(15,6) =".format(n=15, r=6), binomial_coefficient(15, 6))

```

Input:

```

n (Total items) = 15
r (Items required) = 6

```

Output:

```

C(15,6) = 5005

```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 6: AI Project Team Selector

Aim:

To calculate the number of different 3-developer teams that can be formed from a pool of 10 developers for a special AI project.

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size (n+1) x (r+1), initialised to 0.
3. For each row i from 0 to n and each column j from 0 to min(i, r): if j is 0 or j equals i, set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```

def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(10,3) =".format(n=10, r=3), binomial_coefficient(10, 3))

```

Input:

```

n (Total items) = 10
r (Items required) = 3

```

Output:

```

C(10,3) = 120

```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 7: Scholarship Committee Selector

Aim:

To determine the total number of possible 4-member scholarship committees that can be formed from a pool of 12 professors.

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size $(n+1) \times (r+1)$, initialised to 0.
3. For each row i from 0 to n and each column j from 0 to $\min(i, r)$: if j is 0 or j equals i , set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```
def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(12,4) = ".format(n=12, r=4), binomial_coefficient(12, 4))
```

Input:

```
n (Total items) = 12
r (Items required) = 4
```

Output:

```
C(12,4) = 495
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 8: Tournament Squad Selector

Aim:

To calculate the number of possible 5-player tournament squads that can be chosen from a cricket academy of 15 players.

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size $(n+1) \times (r+1)$, initialised to 0.
3. For each row i from 0 to n and each column j from 0 to $\min(i, r)$: if j is 0 or j equals i , set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```
def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(15,5) = ".format(n=15, r=5), binomial_coefficient(15, 5))
```

Input:

```
n (Total items) = 15
r (Items required) = 5
```

Output:

```
C(15,5) = 3003
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 9: Product-Testing Group Selector

Aim:

To determine the total number of 6-customer testing groups that can be formed from 20 volunteers for a new product.

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size (n+1) x (r+1), initialised to 0.
3. For each row i from 0 to n and each column j from 0 to min(i, r): if j is 0 or j equals i, set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```
def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(20,6) = ".format(n=20, r=6), binomial_coefficient(20, 6))
```

Input:

```
n (Total items) = 20
r (Items required) = 6
```

Output:

```
C(20,6) = 38760
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 10: Cloud Server Allocation Counter

Aim:

To determine the number of possible ways to allocate 3 servers to a customer from a pool of 8 available cloud servers.

Algorithm:

1. Read the values of n (total items) and r (items to choose).
2. Create a 2-D table C of size $(n+1) \times (r+1)$, initialised to 0.
3. For each row i from 0 to n and each column j from 0 to $\min(i, r)$: if j is 0 or j equals i , set $C[i][j] = 1$ (base cases).
4. Otherwise set $C[i][j] = C[i-1][j-1] + C[i-1][j]$, using Pascal's Triangle recurrence.
5. The value $C[n][r]$ gives the binomial coefficient, i.e., the number of ways to choose r items from n items.
6. Print $C[n][r]$ as the final answer.

Program (Python Code):

```
def binomial_coefficient(n, r):
    C = [[0] * (r + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(min(i, r) + 1):
            if j == 0 or j == i:
                C[i][j] = 1
            else:
                C[i][j] = C[i - 1][j - 1] + C[i - 1][j]
    return C[n][r]

if __name__ == "__main__":
    print("C(8,3) =".format(n=8, r=3), binomial_coefficient(8, 3))
```

Input:

```
n (Total items) = 8
r (Items required) = 3
```

Output:

```
C(8,3) = 56
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

3. Assembly Line Scheduling

Question 1: Two-Line Product Manufacturing

Aim:

To find the minimum time required to manufacture a product using two assembly lines, each having several stations, with the option to transfer between lines at any station.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays $T1$ and $T2$ of size n (number of stations), where $T1[i]$ / $T2[i]$ store the minimum time to reach station i on Line1 / Line2 respectively.

3. Initialise $T1[0] = \text{entry_time1} + \text{line1}[0]$ and $T2[0] = \text{entry_time2} + \text{line2}[0]$.
4. For each station i from 1 to $n-1$: $T1[i] = \min(T1[i-1] + \text{line1}[i], T2[i-1] + \text{transfer2}[i-1] + \text{line1}[i])$, and symmetrically $T2[i] = \min(T2[i-1] + \text{line2}[i], T1[i-1] + \text{transfer1}[i-1] + \text{line2}[i])$.
5. The minimum overall production time is $\min(T1[n-1] + \text{exit_time1}, T2[n-1] + \text{exit_time2})$.
6. Print this minimum value as the final answer.

Program (Python Code):

```
def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
    n = len(line1)
    T1 = [0] * n
    T2 = [0] * n
    T1[0] = entry[0] + line1[0]
    T2[0] = entry[1] + line2[0]
    for i in range(1, n):
        T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
        T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])
    return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [10, 12]
    exit_ = [18, 7]
    line1 = [4, 5, 3, 2]
    line2 = [2, 10, 1, 4]
    transfer1 = [7, 4, 5]
    transfer2 = [9, 2, 8]
    print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2))
```

Input:

```
Entry Times = [10, 12]
Exit Times = [18, 7]
Line1 = [4, 5, 3, 2]
Line2 = [2, 10, 1, 4]
Transfer1 = [7, 4, 5]
Transfer2 = [9, 2, 8]
```

Output:

```
Minimum Production Time = 35
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 2: Optimal Assembly Line Path Finder

Aim:

To determine the optimal assembly-line path with the minimum total processing time across two parallel lines.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays T1 and T2 of size n (number of stations), where $T1[i]$ / $T2[i]$ store the minimum time to reach station i on Line1 / Line2 respectively.
3. Initialise $T1[0] = \text{entry_time1} + \text{line1}[0]$ and $T2[0] = \text{entry_time2} + \text{line2}[0]$.
4. For each station i from 1 to $n-1$: $T1[i] = \min(T1[i-1] + \text{line1}[i], T2[i-1] + \text{transfer2}[i-1] + \text{line1}[i])$, and symmetrically $T2[i] = \min(T2[i-1] + \text{line2}[i], T1[i-1] + \text{transfer1}[i-1] + \text{line2}[i])$.
5. The minimum overall production time is $\min(T1[n-1] + \text{exit_time1}, T2[n-1] + \text{exit_time2})$.
6. Print this minimum value as the final answer.

Program (Python Code):

```

def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
    n = len(line1)
    T1 = [0] * n
    T2 = [0] * n
    T1[0] = entry[0] + line1[0]
    T2[0] = entry[1] + line2[0]
    for i in range(1, n):
        T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
        T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])
    return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [8, 10]
    exit_ = [12, 15]
    line1 = [5, 6, 4]
    line2 = [7, 3, 5]
    transfer1 = [2, 1]
    transfer2 = [3, 2]
    print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2))

```

Input:

```

Entry Times = [8, 10]
Exit Times  = [12, 15]
Line1 = [5, 6, 4]
Line2 = [7, 3, 5]
Transfer1 = [2, 1]
Transfer2 = [3, 2]

```

Output:

```
Minimum Production Time = 35
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 3: Fastest Route Through Assembly Stations

Aim:

To compute the fastest route through a sequence of assembly stations spread across two production lines.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays T1 and T2 of size n (number of stations), where T1[i] / T2[i] store the minimum time to reach station i on Line1 / Line2 respectively.
3. Initialise T1[0] = entry_time1 + line1[0] and T2[0] = entry_time2 + line2[0].
4. For each station i from 1 to n-1: T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i]), and symmetrically T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i]).
5. The minimum overall production time is min(T1[n-1] + exit_time1, T2[n-1] + exit_time2).
6. Print this minimum value as the final answer.

Program (Python Code):

```

def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
    n = len(line1)
    T1 = [0] * n
    T2 = [0] * n
    T1[0] = entry[0] + line1[0]
    T2[0] = entry[1] + line2[0]
    for i in range(1, n):
        T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
        T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])

```

```

        return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [5, 6]
    exit_ = [4, 5]
    line1 = [3, 4, 5]
    line2 = [4, 3, 6]
    transfer1 = [2, 1]
    transfer2 = [1, 2]
    print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2))

```

Input:

```

Entry Times = [5, 6]
Exit Times  = [4, 5]
Line1 = [3, 4, 5]
Line2 = [4, 3, 6]
Transfer1 = [2, 1]
Transfer2 = [1, 2]

```

Output:

```

Minimum Production Time = 21

```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 4: Minimum-Time Manufacturing Schedule Identifier

Aim:

To identify the assembly-line schedule that minimises the total manufacturing time across two lines with four stations each.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays T1 and T2 of size n (number of stations), where T1[i] / T2[i] store the minimum time to reach station i on Line1 / Line2 respectively.
3. Initialise T1[0] = entry_time1 + line1[0] and T2[0] = entry_time2 + line2[0].
4. For each station i from 1 to n-1: T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i]), and symmetrically T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i]).
5. The minimum overall production time is min(T1[n-1] + exit_time1, T2[n-1] + exit_time2).
6. Print this minimum value as the final answer.

Program (Python Code):

```

def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
    n = len(line1)
    T1 = [0] * n
    T2 = [0] * n
    T1[0] = entry[0] + line1[0]
    T2[0] = entry[1] + line2[0]
    for i in range(1, n):
        T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
        T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])
    return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [7, 9]
    exit_ = [8, 6]
    line1 = [4, 3, 5, 2]
    line2 = [5, 2, 4, 3]
    transfer1 = [1, 2, 1]

```

```

transfer2 = [2, 1, 2]
print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1

```

Input:

```

Entry Times = [7, 9]
Exit Times  = [8, 6]
Line1 = [4, 3, 5, 2]
Line2 = [5, 2, 4, 3]
Transfer1 = [1, 2, 1]
Transfer2 = [2, 1, 2]

```

Output:

Minimum Production Time = 27

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 5: Minimum-Cost Path Through an Assembly System

Aim:

To determine the minimum-cost path through a two-line assembly system with transfer options between stations.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays T1 and T2 of size n (number of stations), where T1[i] / T2[i] store the minimum time to reach station i on Line1 / Line2 respectively.
3. Initialise T1[0] = entry_time1 + line1[0] and T2[0] = entry_time2 + line2[0].
4. For each station i from 1 to n-1: T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i]), and symmetrically T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i]).
5. The minimum overall production time is min(T1[n-1] + exit_time1, T2[n-1] + exit_time2).
6. Print this minimum value as the final answer.

Program (Python Code):

```

def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
    n = len(line1)
    T1 = [0] * n
    T2 = [0] * n
    T1[0] = entry[0] + line1[0]
    T2[0] = entry[1] + line2[0]
    for i in range(1, n):
        T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
        T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])
    return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [6, 7]
    exit_ = [5, 4]
    line1 = [2, 4, 3]
    line2 = [3, 2, 5]
    transfer1 = [1, 2]
    transfer2 = [2, 1]
    print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1

```

Input:

```

Entry Times = [6, 7]
Exit Times  = [5, 4]
Line1 = [2, 4, 3]
Line2 = [3, 2, 5]
Transfer1 = [1, 2]

```

```
Transfer2 = [2, 1]
```

Output:

```
Minimum Production Time = 20
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 6: Automobile Manufacturing Line Scheduler

Aim:

To determine the minimum time required to manufacture a car on an automobile assembly line with two parallel lines of four stations each, given station processing times and inter-line transfer times.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays T1 and T2 of size n (number of stations), where T1[i] / T2[i] store the minimum time to reach station i on Line1 / Line2 respectively.
3. Initialise T1[0] = entry_time1 + line1[0] and T2[0] = entry_time2 + line2[0].
4. For each station i from 1 to n-1: T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i]), and symmetrically T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i]).
5. The minimum overall production time is min(T1[n-1] + exit_time1, T2[n-1] + exit_time2).
6. Print this minimum value as the final answer.

Program (Python Code):

```
def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
    n = len(line1)
    T1 = [0] * n
    T2 = [0] * n
    T1[0] = entry[0] + line1[0]
    T2[0] = entry[1] + line2[0]
    for i in range(1, n):
        T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
        T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])
    return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [10, 12]
    exit_ = [18, 7]
    line1 = [4, 5, 3, 2]
    line2 = [2, 10, 1, 4]
    transfer1 = [7, 4, 5]
    transfer2 = [9, 2, 8]
    print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2))
```

Input:

```
Entry Times = [10, 12]
Exit Times = [18, 7]
Line1 = [4, 5, 3, 2]
Line2 = [2, 10, 1, 4]
Transfer1 = [7, 4, 5]
Transfer2 = [9, 2, 8]
```

Output:

```
Minimum Production Time = 35
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 7: Smartphone Assembly Route Optimiser

Aim:

To determine the optimal route through two smartphone production lines that minimises total production time, given that components may switch lines after any station.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays T1 and T2 of size n (number of stations), where T1[i] / T2[i] store the minimum time to reach station i on Line1 / Line2 respectively.
3. Initialise T1[0] = entry_time1 + line1[0] and T2[0] = entry_time2 + line2[0].
4. For each station i from 1 to n-1: T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i]), and symmetrically T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i]).
5. The minimum overall production time is min(T1[n-1] + exit_time1, T2[n-1] + exit_time2).
6. Print this minimum value as the final answer.

Program (Python Code):

```
def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
    n = len(line1)
    T1 = [0] * n
    T2 = [0] * n
    T1[0] = entry[0] + line1[0]
    T2[0] = entry[1] + line2[0]
    for i in range(1, n):
        T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
        T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])
    return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [8, 10]
    exit_ = [12, 15]
    line1 = [5, 6, 4]
    line2 = [7, 3, 5]
    transfer1 = [2, 1]
    transfer2 = [3, 2]
    print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2))
```

Input:

```
Entry Times = [8, 10]
Exit Times  = [12, 15]
Line1 = [5, 6, 4]
Line2 = [7, 3, 5]
Transfer1 = [2, 1]
Transfer2 = [3, 2]
```

Output:

```
Minimum Production Time = 35
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 8: Food Packaging Line Scheduler

Aim:

To find the minimum completion time for products passing through labelling, sealing, and packing stations on two parallel packaging lines.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays T1 and T2 of size n (number of stations), where T1[i] / T2[i] store the minimum time to reach station i on Line1 / Line2 respectively.
3. Initialise T1[0] = entry_time1 + line1[0] and T2[0] = entry_time2 + line2[0].
4. For each station i from 1 to n-1: T1[i] = min(T1[i-1] + line1[i], T2[i-1] + transfer2[i-1] + line1[i]), and symmetrically T2[i] = min(T2[i-1] + line2[i], T1[i-1] + transfer1[i-1] + line2[i]).
5. The minimum overall production time is min(T1[n-1] + exit_time1, T2[n-1] + exit_time2).
6. Print this minimum value as the final answer.

Program (Python Code):

```
def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
    n = len(line1)
    T1 = [0] * n
    T2 = [0] * n
    T1[0] = entry[0] + line1[0]
    T2[0] = entry[1] + line2[0]
    for i in range(1, n):
        T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
        T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])
    return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [5, 6]
    exit_ = [4, 5]
    line1 = [3, 4, 5]
    line2 = [4, 3, 6]
    transfer1 = [2, 1]
    transfer2 = [1, 2]
    print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2))
```

Input:

```
Entry Times = [5, 6]
Exit Times = [4, 5]
Line1 = [3, 4, 5]
Line2 = [4, 3, 6]
Transfer1 = [2, 1]
Transfer2 = [1, 2]
```

Output:

```
Minimum Production Time = 21
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 9: Medicine Bottle Packaging Scheduler

Aim:

To determine the optimal production schedule for medicine bottles passing through filling, capping, labelling, and packaging stations on two assembly lines.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays T1 and T2 of size n (number of stations), where T1[i] / T2[i] store the minimum time to reach station i on Line1 / Line2 respectively.
3. Initialise T1[0] = entry_time1 + line1[0] and T2[0] = entry_time2 + line2[0].

4. For each station i from 1 to $n-1$: $T1[i] = \min(T1[i-1] + \text{line1}[i], T2[i-1] + \text{transfer2}[i-1] + \text{line1}[i])$, and symmetrically $T2[i] = \min(T2[i-1] + \text{line2}[i], T1[i-1] + \text{transfer1}[i-1] + \text{line2}[i])$.
5. The minimum overall production time is $\min(T1[n-1] + \text{exit_time1}, T2[n-1] + \text{exit_time2})$.
6. Print this minimum value as the final answer.

Program (Python Code):

```
def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
    n = len(line1)
    T1 = [0] * n
    T2 = [0] * n
    T1[0] = entry[0] + line1[0]
    T2[0] = entry[1] + line2[0]
    for i in range(1, n):
        T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
        T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])
    return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [7, 9]
    exit_ = [8, 6]
    line1 = [4, 3, 5, 2]
    line2 = [5, 2, 4, 3]
    transfer1 = [1, 2, 1]
    transfer2 = [2, 1, 2]
    print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2))
```

Input:

```
Entry Times = [7, 9]
Exit Times = [8, 6]
Line1 = [4, 3, 5, 2]
Line2 = [5, 2, 4, 3]
Transfer1 = [1, 2, 1]
Transfer2 = [2, 1, 2]
```

Output:

```
Minimum Production Time = 27
```

Result:

The program was executed successfully and the output was verified to be correct for the given input.

Question 10: Circuit Board Manufacturing Scheduler

Aim:

To find the minimum manufacturing time for circuit boards produced on two parallel assembly lines, where products can switch lines between stations.

Algorithm:

1. Read the entry times, exit times, station processing times for both lines (Line1, Line2), and the transfer times between the lines (Transfer1, Transfer2).
2. Create two 1-D arrays T1 and T2 of size n (number of stations), where $T1[i]$ / $T2[i]$ store the minimum time to reach station i on Line1 / Line2 respectively.
3. Initialise $T1[0] = \text{entry_time1} + \text{line1}[0]$ and $T2[0] = \text{entry_time2} + \text{line2}[0]$.
4. For each station i from 1 to $n-1$: $T1[i] = \min(T1[i-1] + \text{line1}[i], T2[i-1] + \text{transfer2}[i-1] + \text{line1}[i])$, and symmetrically $T2[i] = \min(T2[i-1] + \text{line2}[i], T1[i-1] + \text{transfer1}[i-1] + \text{line2}[i])$.
5. The minimum overall production time is $\min(T1[n-1] + \text{exit_time1}, T2[n-1] + \text{exit_time2})$.
6. Print this minimum value as the final answer.

Program (Python Code):

```
def assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2):
```



```

n = len(line1)
T1 = [0] * n
T2 = [0] * n
T1[0] = entry[0] + line1[0]
T2[0] = entry[1] + line2[0]
for i in range(1, n):
    T1[i] = min(T1[i - 1] + line1[i], T2[i - 1] + transfer2[i - 1] + line1[i])
    T2[i] = min(T2[i - 1] + line2[i], T1[i - 1] + transfer1[i - 1] + line2[i])
return min(T1[n - 1] + exit_[0], T2[n - 1] + exit_[1])

if __name__ == "__main__":
    entry = [6, 7]
    exit_ = [5, 4]
    line1 = [2, 4, 3]
    line2 = [3, 2, 5]
    transfer1 = [1, 2]
    transfer2 = [2, 1]
    print("Minimum Production Time =", assembly_line_scheduling(entry, exit_, line1, line2, transfer1, transfer2))

```

Input:

```

Entry Times = [6, 7]
Exit Times  = [5, 4]
Line1 = [2, 4, 3]
Line2 = [3, 2, 5]
Transfer1 = [1, 2]
Transfer2 = [2, 1]

```

Output:

```

Minimum Production Time = 20

```

Result:

The program was executed successfully and the output was verified to be correct for the given input.